



Decision Console - マーケ施策意思決定は残す認識で合っていますか？
そうでなければ、それでおしえて。
合っていれば、完全にコード（WolframONEも含めた）まで入れた詳細なClaudeに渡すものを作成して。



Thought for 1m 0s >

結論から言うと、「**Decision Console - マーケ施策意思決定**」は**残す前提で OK**です。
役割は「経営・マーケ責任者向けのトップ画面（Exec Overview）」に振り切る、という整理でいきましょう。
以下、その前提で **Claude** にそのまま渡せる仕様＋コードスケルトン をまとめます。
（既存コードへの影響を最小にしつつ、「v2 ビュー」を追加する形です）

Claude 用仕様書

「Decision Console v2（マーケ施策意思決定）」実装指示

0. 前提・環境

- バックエンド: FastAPI (Python 3.x, Uvicorn)
- フロントエンド: React + Vite + TypeScript + TanStack Query
- UI: 既存のダークテーマ (CQOx UI) を踏襲
- i18n: 既に EN/日本語トグルあり (lang 状態で文言切り替え)
- 既存の Decision Console v1 は URL と API を壊さず残す (互換性維持)
 - v2 は /console (新 UI) とし、v1 は /decision-console-v1 などへ温存か、もしくは Feature Flag で切り替え

1. 目的 / 非目的

目的

- 経営・マーケ責任者が 1 画面で「今のマーケ施策ポートフォリオの状態」を判断できる こと
- すべての指標を 因果推定 (Δ¥、CAS、リスク) と紐づける こと
 - 単なる CVR / 売上ダッシュボードではなく、「インクリメンタルな貢献」が見える
- セグメント / チャネル / ポリシーごとに ドリルダウン可能な入口 を提供すること
 - 詳細は Diagnostics / Portfolio / Policy Lab 側で見る

非目的

- BI ツールのように「自由に何でもグラフを作れる」セルフサービスを提供すること
- モデル学習や因果診断そのものの UI をここで完結させること
（それらは既存の Causal Design / Diagnostics で行う）
- 既存 API スキーマを破壊すること

2. 画面レイアウト仕様 (Decision Console v2)

2.1 レイアウト概要

3 段構成:

- サマリタイトル (KPI 概要)
- 中央: 時間 / セグメント / チャネルの 3 ビュー
- 下段: Decision Cards (ポリシー別の意思決定一覧)

2.1.1 上段: サマリタイトル

4 つのカード (左から):

- Total Incremental Profit (合計インクリメンタル利益 Δ¥)

- 2. **Average Δ¥ per Policy** (施策あたり平均 Δ¥)
- 3. **Mean CAS Score** (品質スコア平均)
- 4. **Risk (CVaR@p)**
 - 「一定確率でこれくらい損する」インクリメンタル損失の期待値 (負の Δ¥)

各カード共通仕様：

- 表示値：数値 + 単位
- サブテキスト：
 - Δ¥ 系: 対目標比 (%)
 - CAS: 閾値と比較 (例: `Target ≥ 0.75`)
 - Risk: 許容リスクレンジと比較

2.1.2 中段左：Δ¥ 推移 (週次)

- x: 週 (例: `2025-W45`)
- y: 合計 Δ¥ (円)
- シリーズ：
 - 実績 Δ¥ (棒 or 線)
 - 目標 Δ¥ (点線)
- クリックでその週の Decision Cards にフィルタを適用

2.1.3 中段中央：セグメント BCG 風散布図

- x 軸: セグメント規模 (対象ユーザ数 or 売上)
- y 軸: セグメント別 Δ¥ (インクリメンタル利益)
- 点のサイズ: 施策数 or 投下予算
- 点の色: CAS (高品質=緑, 低品質=赤)
- 四象限に薄いガイド線 + ラベル：
 - Stars / Question / Cash Cow / Dogs (文言は i18n)

2.1.4 中段右：チャンネル別パフォーマンス

- チャンネルごとの棒グラフ
 - x: チャンネル名 (Email / App Push / SNS / Search / Display ...)
 - y1: Δ¥ (棒)
 - y2: ROI (折れ線) ※第二軸
- 棒クリックで Decision Cards をチャンネルでフィルタ

2.1.5 下段：Decision Cards (施策別一覧)

テーブル or カードリスト：

- 列：
 - Policy Name
 - Δ¥ (円)
 - ROI
 - CAS
 - Risk Score
 - Verdict (Go / Canary / Hold)
 - 時期 (週 or 日付レンジ)
- テーブル上部にフィルタ：
 - Verdict (Go / Canary / Hold) トグル
 - 期間 (DateRange)
 - セグメント / チャンネル (セレクト)

3. API / データモデル仕様

3.1 新規エンドポイント

`GET /api/v1/decision-console/overview`

クエリパラメータ：

- `start_date: YYYY-MM-DD`

- `end_date: YYYY-MM-DD`
- `locale: "ja" | "en"` (文言タグには使うが、データは同じ)

レスポンス JSON スキーマ (簡略) :

```
{
  "summary": {
    "total_incremental_profit": 36800000.0, // 円
    "average_delta": 2450000.0, // 円
    "mean_cas": 0.87, // 0-1
    "risk_cvar": -5200000.0 // 円 (損失なのでマイナス想定)
  },
  "time_series": {
    "weekly_delta": [
      { "week": "2025-W45", "delta_yen": 1200000.0, "target_yen": 1000000.0 },
      ...
    ]
  },
  "segments": [
    {
      "segment_id": "seg_high_value_urban",
      "segment_name": "High Value Urban",
      "population": 150000,
      "delta_yen": 5200000.0,
      "policy_count": 3,
      "cas_mean": 0.9
    }
  ],
  "channels": [
    {
      "channel": "email",
      "delta_yen": 18000000.0,
      "roi": 4.5
    }
  ],
  "policies": [
    {
      "policy_id": "pol-001",
      "policy_name": "Dynamic Discount Policy",
      "segment_id": "seg_high_value_urban",
      "channel": "email",
      "delta_yen": 2450000.0,
      "roi": 2.8,
      "cas": 0.87,
      "risk_score": 0.12,
      "verdict": "go", // "go" | "canary" | "hold"
      "start_date": "2025-11-01",
      "end_date": "2025-11-28"
    }
  ]
}
```

※ 既存の分析結果テーブル (例: `analyses`, `policies`, `segments`) から集計する想定。
正確なテーブル名はリポジトリに合わせて Claude 側でマッピングする。

4. バックエンド実装 (FastAPI)

4.1 ルータファイル例

`backend/api/decision_console_v2.py` (新規)

```
from datetime import date
from typing import List, Literal, Optional

from fastapi import APIRouter, Depends, Query
from pydantic import BaseModel

# ここは既存の依存関係解決に合わせて調整
from .deps import get_db_session
```

```

from .services.decision_console import (
    get_summary_metrics,
    get_weekly_delta_series,
    get_segment_portfolio_points,
    get_channel_performance,
    get_policy_cards,
)

router = APIRouter(prefix="/api/v1/decision-console", tags=["decision-console"])

class SummaryModel(BaseModel):
    total_incremental_profit: float
    average_delta: float
    mean_cas: float
    risk_cvar: float

class WeeklyDeltaPoint(BaseModel):
    week: str
    delta_yen: float
    target_yen: float

class SegmentPoint(BaseModel):
    segment_id: str
    segment_name: str
    population: int
    delta_yen: float
    policy_count: int
    cas_mean: float

class ChannelPerformance(BaseModel):
    channel: str
    delta_yen: float
    roi: float

class PolicyCard(BaseModel):
    policy_id: str
    policy_name: str
    segment_id: Optional[str]
    channel: Optional[str]
    delta_yen: float
    roi: float
    cas: float
    risk_score: float
    verdict: Literal["go", "canary", "hold"]
    start_date: date
    end_date: date

class DecisionConsoleOverview(BaseModel):
    summary: SummaryModel
    time_series: dict
    segments: List[SegmentPoint]
    channels: List[ChannelPerformance]
    policies: List[PolicyCard]

@router.get("/overview", response_model=DecisionConsoleOverview)
def get_decision_console_overview(
    start_date: date = Query(...),
    end_date: date = Query(...),
    locale: str = Query("ja"),
    db=Depends(get_db_session),
) -> DecisionConsoleOverview:
    """

```

```

Decision Console v2 用の集約データを返す。
既存の分析結果テーブルから集計するだけで、新たな学習は行わない。
"""
summary = get_summary_metrics(db, start_date, end_date)
weekly = get_weekly_delta_series(db, start_date, end_date)
segments = get_segment_portfolio_points(db, start_date, end_date)
channels = get_channel_performance(db, start_date, end_date)
policies = get_policy_cards(db, start_date, end_date)

return DecisionConsoleOverview(
    summary=summary,
    time_series={"weekly_delta": weekly},
    segments=segments,
    channels=channels,
    policies=policies,
)

```

4.2 リスク (CVaR) 計算ロジック (Python)

backend/services/decision_console.py に追加：

```

import numpy as np
from typing import Sequence

def compute_cvar(loss_samples: Sequence[float], alpha: float = 0.95) -> float:
    """
    損失分布サンプルから CVaR を計算する。
    loss_samples: Δ% の損失 (マイナス値) サンプル
    alpha: 信頼水準 (0.95 なら最悪 5% の平均損失)

    戻り値: CVaR (円, 負の値が大きいほどリスク大)
    """
    if not loss_samples:
        return 0.0
    losses = np.sort(np.array(loss_samples))
    cutoff_idx = int(np.floor(alpha * len(losses)))
    tail_losses = losses[cutoff_idx:]
    return float(np.mean(tail_losses)) if len(tail_losses) > 0 else float(losses[-1])

```

※ 実際には各ポリシーのシミュレーション / ブートストラップ結果から損失サンプルを取得する想定。
サンプルが無い場合は 0 を返すか、`None` を返してフロントで「N/A」と表示する。

5. Wolfram One 連携 (任意・高度オプション)

ポートフォリオ最適化や高度なリスク計算を Wolfram にオフロードするための例。
あくまで オプション であり、Wolfram が利用できない環境では従来ロジックで代替する。

```

from typing import List, Tuple

try:
    from wolframclient.evaluation import WolframLanguageSession
    from wolframclient.language import wl, wlexpr
except ImportError:
    WolframLanguageSession = None

def compute_efficient_frontier_with_wolfram(
    returns: List[float], risks: List[float]
) -> List[Tuple[float, float]]:
    """
    Wolfram Language を使って簡易的な効率フロンティアを計算する例。
    returns: 各ポリシーの期待 Δ%
    risks: 各ポリシーのリスク (標準偏差など)

    戻り値: [(risk, return), ...] フロンティア上の点
    """
    if WolframLanguageSession is None:
        # フォールバック: 既存ロジック or 空

```

```

return []

with WolframLanguageSession() as session:
    data = list(zip(risks, returns))
    # 単純な ConvexHull をフロンティアとして扱う一例
    expr = wl.ConvexHullMesh(data)
    frontier_points = session.evaluate(wl.MeshCoordinates(expr))
    # frontier_points は [[r1, ret1], [r2, ret2], ...] 形式を想定
    return [(float(r), float(ret)) for r, ret in frontier_points]

```

※ 実運用では、ここから更にポートフォリオ最適化（制約付き最大化）へ発展可能。

6. フロントエンド実装 (React + TS)

6.1 型定義

frontend/src/types/decisionConsole.ts (新規)

```

export type Verdict = "go" | "canary" | "hold";

export interface Summary {
    total_incremental_profit: number;
    average_delta: number;
    mean_cas: number;
    risk_cvar: number;
}

export interface WeeklyDeltaPoint {
    week: string;
    delta_yen: number;
    target_yen: number;
}

export interface SegmentPoint {
    segment_id: string;
    segment_name: string;
    population: number;
    delta_yen: number;
    policy_count: number;
    cas_mean: number;
}

export interface ChannelPerformance {
    channel: string;
    delta_yen: number;
    roi: number;
}

export interface PolicyCard {
    policy_id: string;
    policy_name: string;
    segment_id?: string;
    channel?: string;
    delta_yen: number;
    roi: number;
    cas: number;
    risk_score: number;
    verdict: Verdict;
    start_date: string;
    end_date: string;
}

export interface DecisionConsoleOverview {
    summary: Summary;
    time_series: {
        weekly_delta: WeeklyDeltaPoint[];
    };
    segments: SegmentPoint[];
    channels: ChannelPerformance[];
}

```

```

    policies: PolicyCard[];
  }

```

6.2 API フック

frontend/src/api/decisionConsole.ts :

```

import { useQuery } from "@tanstack/react-query";
import { DecisionConsoleOverview } from "../types/decisionConsole";

export function useDecisionConsoleOverview(params: {
  startDate: string;
  endDate: string;
  locale: "ja" | "en";
}) {
  const { startDate, endDate, locale } = params;

  return useQuery<DecisionConsoleOverview>({
    queryKey: ["decision-console-overview", startDate, endDate, locale],
    queryFn: async () => {
      const search = new URLSearchParams({
        start_date: startDate,
        end_date: endDate,
        locale,
      });
      const res = await fetch(`/api/v1/decision-console/overview?${search.toString()}`);
      if (!res.ok) {
        throw new Error("Failed to fetch decision console overview");
      }
      return res.json();
    },
  });
}

```

6.3 メインページコンポーネント

frontend/src/pages/DecisionConsolePageV2.tsx :

```

import React, { useState } from "react";
import { useDecisionConsoleOverview } from "../api/decisionConsole";
import {
  DecisionConsoleOverview as Overview,
  PolicyCard,
} from "../types/decisionConsole";
import { WeeklyDeltaChart } from "../components/decision-console/WeeklyDeltaChart";
import { SegmentPortfolioChart } from "../components/decision-console/SegmentPortfolioChart";
import { ChannelPerformanceChart } from "../components/decision-console/ChannelPerformanceChart";
import { PolicyTable } from "../components/decision-console/PolicyTable";

interface Props {
  defaultStartDate: string;
  defaultEndDate: string;
  locale: "ja" | "en";
}

export const DecisionConsolePageV2: React.FC<Props> = ({
  defaultStartDate,
  defaultEndDate,
  locale,
}) => {
  const [startDate, setStartDate] = useState(defaultStartDate);
  const [endDate, setEndDate] = useState(defaultEndDate);
  const [selectedWeek, setSelectedWeek] = useState<string | null>(null);
  const [selectedChannel, setSelectedChannel] = useState<string | null>(null);
  const [verdictFilter, setVerdictFilter] = useState<"all" | "go" | "canary" | "hold">("all");

  const { data, isLoading, error } = useDecisionConsoleOverview({
    startDate,

```

```

    endDate,
    locale,
  });

  if (isLoading) return <div>Loading Decision Console...</div>;
  if (error || !data) return <div>Failed to load Decision Console.</div>;

  const filteredPolicies = filterPolicies(data, {
    selectedWeek,
    selectedChannel,
    verdictFilter,
  });

  return (
    <div className="p-6 space-y-6">
      { /* 上部: 期間セレクト + サマリカード */ }
      <header className="flex flex-col gap-4">
        { /* 期間セレクト (詳細は既存の DateRange コンポーネントを利用) */ }
        { /* Summary Cards */ }
        <SummaryCards summary={data.summary} locale={locale} />
      </header>

      { /* 中段: 3 つのチャート */ }
      <section className="grid grid-cols-1 xl:grid-cols-3 gap-4">
        <WeeklyDeltaChart
          data={data.time_series.weekly_delta}
          locale={locale}
          onSelectWeek={setSelectedWeek}
        />
        <SegmentPortfolioChart segments={data.segments} />
        <ChannelPerformanceChart
          channels={data.channels}
          onSelectChannel={setSelectedChannel}
        />
      </section>

      { /* 下段: Policy Table */ }
      <section>
        <PolicyTable
          policies={filteredPolicies}
          verdictFilter={verdictFilter}
          onChangeVerdictFilter={setVerdictFilter}
          locale={locale}
        />
      </section>
    </div>
  );
};

function filterPolicies(
  overview: Overview,
  opts: {
    selectedWeek: string | null;
    selectedChannel: string | null;
    verdictFilter: "all" | "go" | "canary" | "hold";
  }
): PolicyCard[] {
  return overview.policies.filter((p) => {
    if (opts.selectedChannel && p.channel !== opts.selectedChannel) return false;
    if (opts.verdictFilter !== "all" && p.verdict !== opts.verdictFilter) return false;
    if (opts.selectedWeek) {
      // 週単位フィルタの簡易例: 実装では start/end_date を週番号に変換して判定する
      // ここは Claude 側で補完調整
    }
    return true;
  });
}

```

チャートコンポーネント（`WeeklyDeltaChart`、`SegmentPortfolioChart`、`ChannelPerformanceChart`）は `recharts` 等で実装。ここでは詳細は割愛し、「棒 + 線」「散布図」「棒 + 折れ線」の構成とだけ指定。

7. シェルコマンド / ラン・テスト

ファイル作成・編集

```
# ブランチ作成
git checkout -b feature/decision-console-v2

# 型・API・ページ用ディレクトリ
mkdir -p frontend/src/types
mkdir -p frontend/src/api
mkdir -p frontend/src/pages
mkdir -p frontend/src/components/decision-console

# バックエンド用ディレクトリ
mkdir -p backend/api
mkdir -p backend/services
```

※ 実際のパスは既存構成に合わせて調整すること。

起動・テスト

```
# backend
cd backend
uvicorn app.main:app --reload

# frontend
cd frontend
npm run dev

# backend テスト (例)
pytest tests/api/test_decision_console_v2.py -q
```

`tests/api/test_decision_console_v2.py` では：

- `/api/v1/decision-console/overview` が 200 を返す
- `summary / segments / policies` のフィールドが JSON スキーマ通り
- `start/end_date` のフィルタが効いている

ことを検証。

8. 「What this does」（この変更で何ができるか）

1. **Decision Console v2** を追加し、経営・マーケ責任者向けに「因果ベースの KPI」を 1 画面で俯瞰できるようにします。
 - Total Δ ¥ / 平均 Δ ¥ / CAS 平均 / CVaR をサマリ表示。
2. **Δ ¥ 推移（週次）**、セグメント **BCG 風** 散布図、チャネル別パフォーマンスの 3 つの視点から施策ポートフォリオを可視化します。
 - どのセグメント・チャネルに張るべきかを直感的に判断可能。
3. **Decision Cards（施策テーブル）** をフィルタ付きで提供し、「どの施策を **Go / Canary / Hold** するか」を具体的な数字とともに示します。
 - v1 のページは温存しつつ、v2 を世界水準の意思決定ビューとして追加。

9. Expert insight (Google/Meta/NASA レベル)

ダッシュボードは「観測用」ではなく「意思決定用」に設計する。

- 世界レベルのチームでは、ダッシュボードの評価軸は「どれだけの意思決定が、この画面だけで完結するか」です。
- そのためには、**KPI (Δ ¥ / CAS / リスク) → セグメント / チャネル → 具体施策 (Decision Card)** という 一方向のストーリー を 1 画面に埋め込むことが重要で、ここで提案した Decision Console v2 の構造は、そのストーリーを最小のクリック数で辿れるように設計しています。

上記をそのまま Claude に渡してもらえれば、

「既存の CQOx リポジトリに Decision Console v2 を追加する」というタスクとして十分具体的に動けるはずです。